

Mutex and Semaphores:

Race Conditions on Shared Hardware

Queues are great for passing data packets. But what if two tasks need to control the exact same physical hardware component?

Imagine we are building an automation system where:

- Task 1 writes a long logging message to an I2C LCD Display or the Serial Monitor.
- Task 2 reads an error and tries to write an alert message to that *exact same* Display or Serial Monitor.

If Task 1 gets interrupted halfway through printing by Task 2, the output gets completely garbled (e.g., [Tas[ALERTask 1] k 2] Ok). This happens because the hardware resource has no lock on it.

FreeRTOS solves this using a Mutex (Mutual Exclusion). Think of a Mutex as a single physical key to a locked room.

- If a task wants to use the resource, it must Take the key using `xSemaphoreTake()`.
- If another task tries to use the resource while the key is gone, the scheduler forces it to sleep (Block) and wait.
- Once the first task is done, it Gives the key back using `xSemaphoreGive()`

Code: (Without Semaphore)

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"

SemaphoreHandle_t serial_mutex;

void send_secure_message(const char *name, const char *status){
    printf("[%s] Starting transmission \n", name);
    vTaskDelay(10 / portTICK_PERIOD_MS);
    printf("Status is: %s \n", status);
}

void task_alpha(void *pvParameters){
    while(1){
        send_secure_message("Task Alpha", "Running");
        vTaskDelay(150 / portTICK_PERIOD_MS);
    }
}
```

Output:

The data streams are heavily interleaved. Both tasks execute simultaneously, printing their starting tags sequentially before completing their status print lines, resulting in mixed data outputs.

Code: (With Semaphore)

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"

SemaphoreHandle_t serial_mutex;

void send_secure_message(const char *name, const char *status){

    if(xSemaphoreTake(serial_mutex, portMAX_DELAY) == pdTRUE){
        printf("[%s] Starting transmission \n", name);
        vTaskDelay(10 / portTICK_PERIOD_MS);
        printf("Status is: %s \n", status);
        xSemaphoreGive(serial_mutex);
    }
}

void task_alpha(void *pvParameters){
    while(1){
        send_secure_message("Task Alpha", "Running");
        vTaskDelay(150 / portTICK_PERIOD_MS);
    }
}

void task_beta(void *pvParameters){
    while(1){
        send_secure_message("Task Beta", "Critical_Error");
        vTaskDelay(150 / portTICK_PERIOD_MS);
    }
}

void app_main(void){
    serial_mutex = xSemaphoreCreateMutex();

    xTaskCreate(task_alpha, "Alpha", 2048, NULL, 5, NULL);
    xTaskCreate(task_beta, "Beta", 2048, NULL, 5, NULL);
}
```

Output:

```
PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS ESP-IDF
I (237) spi_flash: flash io: dio
W (240) spi_flash: Detected size(4096k) larger than the size in the binary image header(2048k). Using the size in the binary image
I (253) main_task: Started on CPU0
I (253) main_task: Calling app_main()
[Task Alpha] Starting transmission
I (253) main_task: Returned from app_main()
Status is: Running
[Task Beta] Starting transmission
Status is: Critical_Error
[Task Alpha] Starting transmission
Status is: Running
[Task Beta] Starting transmission
Status is: Critical_Error
[Task Alpha] Starting transmission
Status is: Running
[Task Beta] Starting transmission
Status is: Critical_Error
[Task Alpha] Starting transmission
Status is: Running
[Task Beta] Starting transmission
Status is: Critical_Error
[Task Alpha] Starting transmission
Status is: Running
[Task Beta] Starting transmission
Status is: Critical_Error
[Task Alpha] Starting transmission
Status is: Running
[Task Beta] Starting transmission
Status is: Critical_Error
[Task Alpha] Starting transmission
Status is: Running
[Task Beta] Starting transmission
Status is: Critical_Error
[Task Alpha] Starting transmission
Status is: Running
```

Complete isolation. Each task acquires the serial lock, completes its transmission atomically, and fully unlocks the hardware resource before the next task can interject.

The Conflict (No Lock Control)

Without lock implementations, task threads cut in midway through data output processes:

[Task Alpha] Starting transmission

[Task Beta] Starting transmission

Status is: Running

Status is: Critical_Error

The Solution (Mutex Integration APIs)

- `xSemaphoreCreateMutex()`: Spawns an atomic binary lock engine.
- `xSemaphoreTake(mutex, timeout)`: Obtains exclusive ownership rights. If the key is checked out, the requesting thread halts.
- `xSemaphoreGive(mutex)`: Returns the token, unblocking subsequent waiting threads.